

Documentation technique



Image Pixabay

Réflexion initiale sur les technologies utilisées :

L'application EcoRide repose sur une stack technique volontairement légère et maîtrisable

Technologies choisies :

Front End :

- **HTML5 / CSS3 / Bootstrap 5** : structure des pages et mise en forme responsive.
- **JavaScript** : interactivité (fetch), chargement dynamique de page, système de routage dynamique, gestion des événements, création de formulaires dynamiques.
- **Chart.js** : pour l'affichage des statistiques graphiques.

Back End :

-**PHP vanilla** : J'ai choisi PHP vanilla plutôt qu'un framework afin de mieux comprendre le fonctionnement natif du langage, maîtriser entièrement le traitement des requêtes HTTP, la gestion des sessions, des échanges JSON et l'accès aux bases de données. Ce choix me permet de conserver un contrôle total sur la logique applicative et de construire une architecture claire adaptée au projet d'examen, tout en évitant la complexité supplémentaire qu'imposerait un framework complet.

L'application est organisée selon une structure inspirée du modèle MVC simplifié. Les contrôleurs back-end PHP reçoivent les requêtes envoyées par le contrôleur front-end JavaScript via `fetch()`, vérifient la méthode HTTP, contrôlent les droits utilisateur et appliquent les règles de sécurité avant d'appeler les fichiers du modèle. Cette séparation permet de distinguer clairement la gestion de l'expérience utilisateur côté navigateur et la sécurisation des traitements côté serveur.

-**PDO (PHP Data Objects)** est utilisé pour l'accès aux bases de données MySQL. Il permet d'exécuter des requêtes préparées avec paramètres bindés, limitant fortement les risques d'injection SQL et assurant une gestion fiable des connexions. Toutes les opérations sensibles utilisent ce mécanisme sécurisé.

-**POO** : Afin d'améliorer la maintenabilité, la lisibilité et l'évolutivité du projet, une structuration partielle en Programmation Orientée Objet a été introduite. La classe `Covoiturage` a notamment été créée pour encapsuler les informations liées à un trajet (départ, arrivée, horaire, état, prix, véhicule, etc.) et fournir des méthodes métier spécifiques, comme la détection d'un trajet écologique selon le type de véhicule.

Cette classe est utilisée dans le fichier `get_covoiturages.php` afin de transformer les résultats SQL en objets puis en tableaux via une méthode `toArray()`, facilitant ainsi le retour des données au format JSON vers le front-end. Elle est placée dans un dossier dédié `Classes` et importée via `require_once`. Par contrainte de temps dans le cadre de l'examen, seule cette fonctionnalité a été structurée en POO, mais cette organisation pourrait être étendue dans une version future aux autres entités comme `Utilisateur` ou `Réservation`.

Bases de données :

- **MySQL (relationnelle)** : pour la gestion des données principales : utilisateurs, covoiturages, véhicules, réservation, préférences.
- **MongoDB (NoSQL)** : pour logs de la plateforme (avis, crédits, suspensions...) et me baser sur ces logs pour les statistiques.

Déploiement :

- **Heroku** : hébergement du site PHP avec base de données MySQL (via **JawsDB**) et MongoDB (via **Atlas**).
- **Composer** : gestion des dépendances PHP (phpmailer, mongodb, dotenv).
- **Docker** : conteneurisation du projet avec **Docker** et **Docker Compose** pour un déploiement en local.

Configuration de l'environnement de travail :

Local :

- **XAMPP** : Apache + MySQL pour le serveur local et phpMyAdmin pour la gestion de la base de données.
- **MongoDB Compass** : avec l'ensemble des collections logs NoSQL.
- **Visual Studio Code (VSCode) comme IDE avec les extensions** :
 1. GitGraph pour le suivi des commits,
 2. Live Sass Compiler pour les compilations SCSS en CSS (utilisé pour personnaliser Bootstrap),
 3. PHP Intelephense et PHP Namespace Resolver pour programmer et importer les classes plus facilement,
 4. MongoDB pour VSCode me permettant d'accéder aux collections depuis l'IDE,
 5. PHP Server pour ouvrir un serveur local et PHPUnit pour les tests.
- **Docker (optionnel)** :
 - Sur Windows et Mac, Docker Desktop intègre automatiquement Docker Compose.
 - Sur Linux, Docker Engine et Docker Compose doivent être installés séparément.

Le projet devra donc contenir les fichiers suivants liés à Docker :

1. **Dockerfile** : définit l'image Apache et PHP avec les modules nécessaires (pdo_mysql, etc.).
2. **docker-compose.yml** : orchestre les services web, MySQL et MongoDB avec des volumes persistants.
3. **.dockerignore** : exclut les fichiers inutiles de la construction de l'image.
4. **.htaccess** : permet au routeur JavaScript (SPA) de rediriger toutes les requêtes vers index.php.

Dépendances installées via Composer :

- **phpmailer/phpmailer (v6.9)** : pour l'envoi de mails,
- **mongodb/mongodb (v1.20)** : client MongoDB natif PHP,
- **vlucas/phpdotenv (v5.6)** : gestion sécurisée des variables d'environnement.

Environnement distant :

- **Git via GitHub** : pour versionner mon projet, travailler sur différentes branches (main, dev, feature/*) et revenir sur des versions précédentes quand nécessaire.
- **Heroku** : déploiement de l'application (connecté à GitHub).
- **JawsDB MySQL** : add-on de Heroku pour la base de données relationnelle.
- **MongoDB Atlas** : base de données NoSQL distante pour les logs.
- **MySQL Workbench** : interface pour gérer JawsDB MySQL

Schémas MCD de la base de données relationnelle

(réalisés sur dbdiagram.io)



Diagramme de classe (via draw.io) :

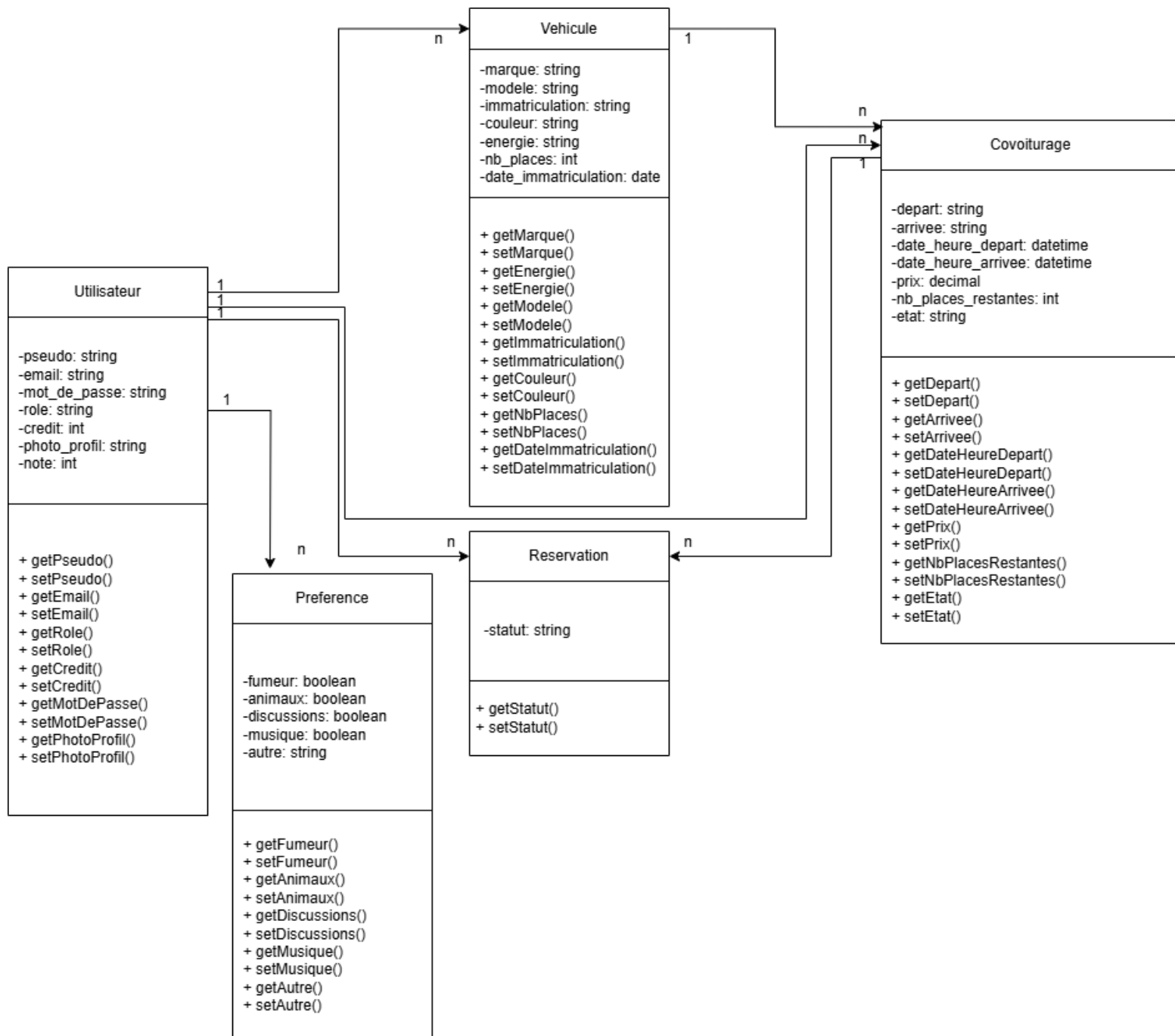
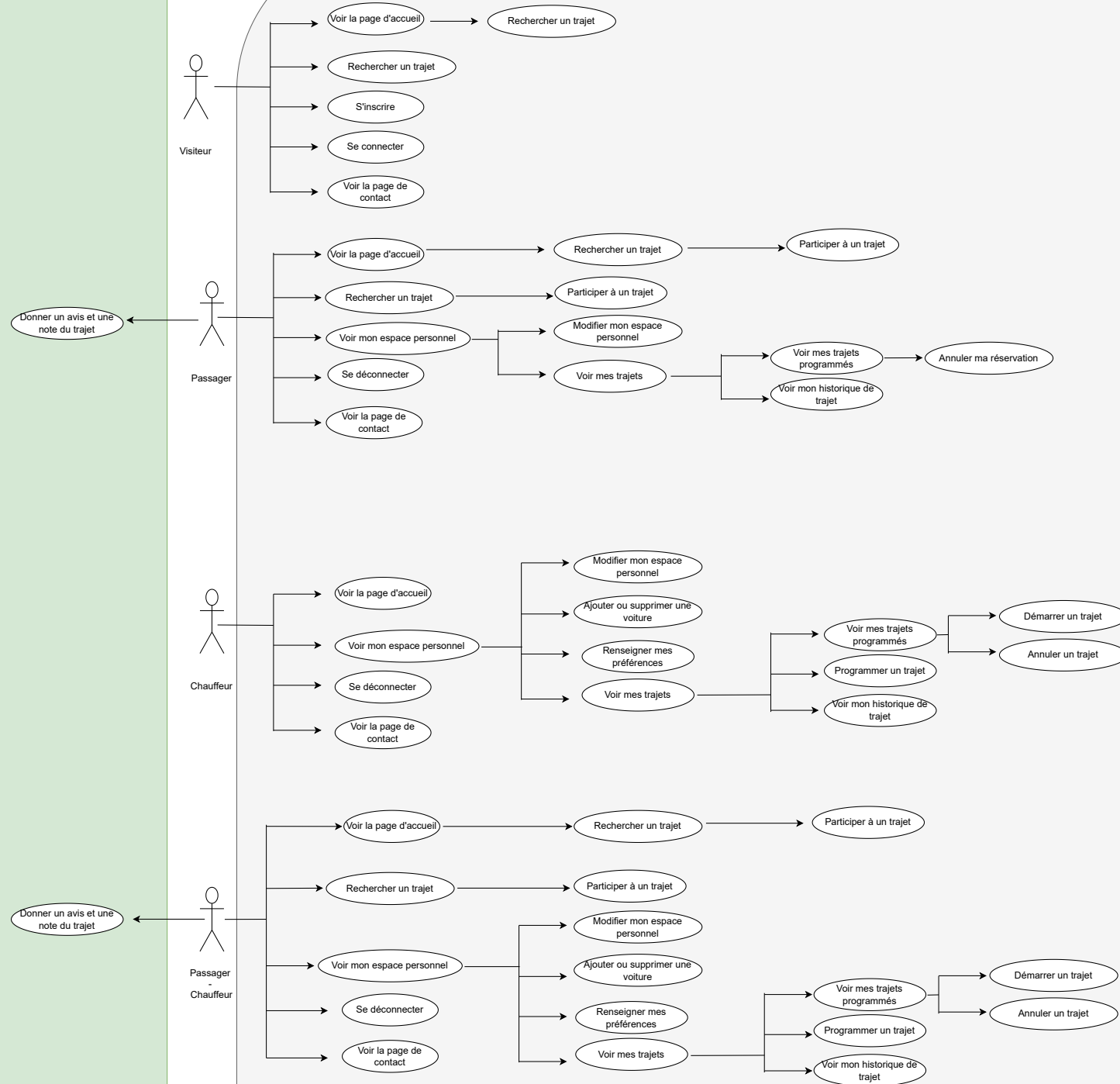


Diagramme d'utilisation via draw.io (insertion du PDF page suivante):

Email

Application Ecoride



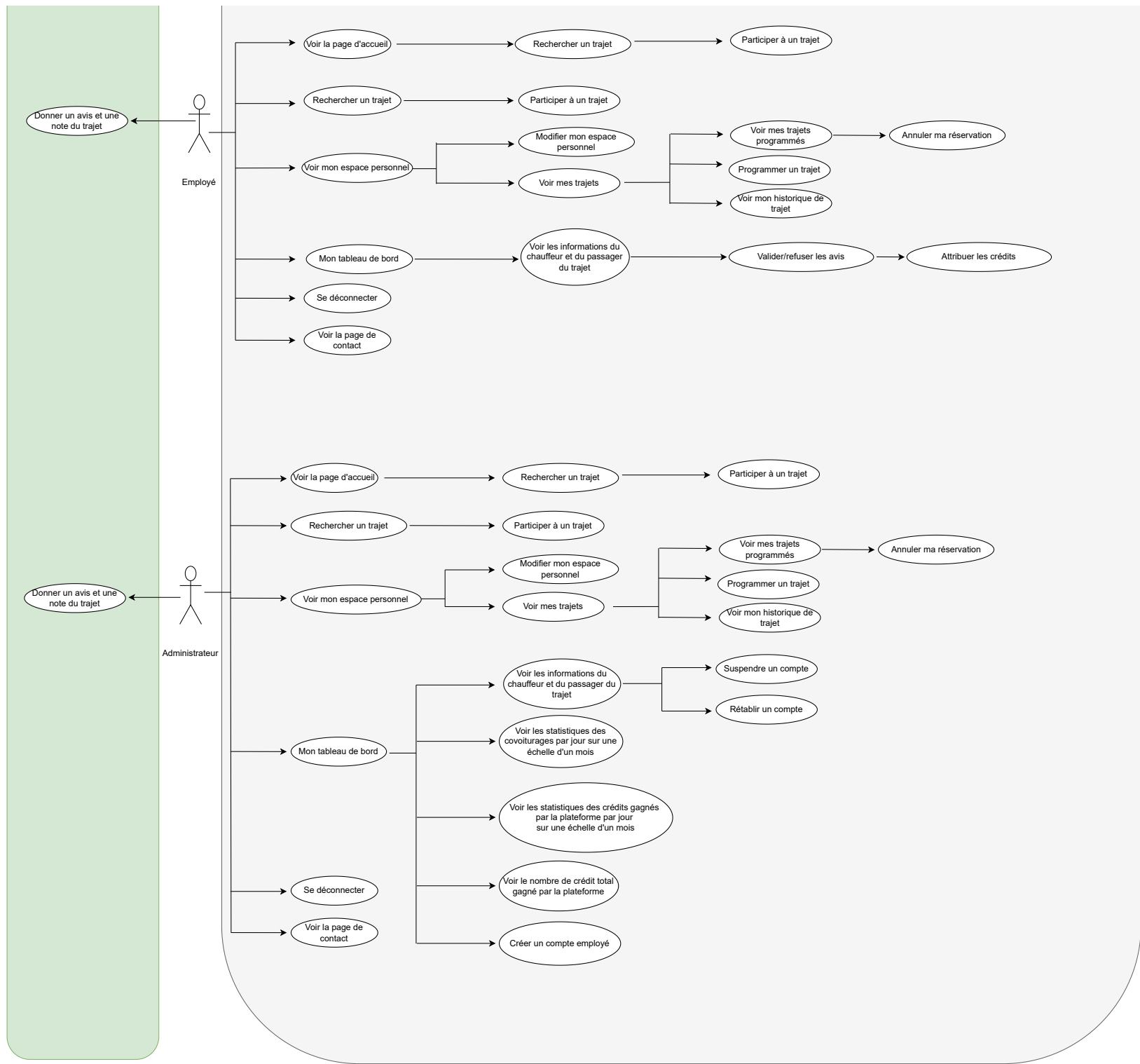


Diagramme de séquence via draw.io (insertion du PDF page suivante) :



Employé

Validation / Refus avis

Interface Vue

espace_employe.php

Controleur Front

employe.js

Controleur Back

gestion_avis_controller.php

Modèle

gestion_avis.php

Base de données

MySQL

MongoDB

MySQL

MongoDB

MySQL

Clique sur le bouton "Valider" ou "Refuser"

Transmet l'information que le bouton "Valider" ou "Refuser" a été cliqué

Affichage du prompt

Si clique sur "Refuser" on affiche un prompt pour inviter l'employé à rentrer le nombre de crédits à attribuer au chauffeur.

Saisit la valeur souhaitée

Transmission des valeurs au controleur.

Envoie une requête fetch() POST vers

Appel PHP

gestion_avis.php
Valide les paramètres

Requête SQL pour récupérer le prix du trajet

Récupère l'avis dans la collection avis_trajet

Réception des réponses MySQL et MongoDB

Attribution des crédits au chauffeur s'il y en a 2 crédits sont retenus pour la plateforme.

Met à jour l'avis sur "validé" ou "refusé" dans la collection avis_trajet.
Ajout de l'enregistrement de l'action dans la collection logs_employe.

gestion_avis.php
Calcule la moyenne avec les avis 'validé' dans la collection MongoDB avis_trajet

Met à jour le champ note du chauffeur

Affiche l'alerte

Envoi une alerte pour avertir du succès ou de l'échec

Envoi le succès ou l'échec de l'action

Retour PHP



Utilisateur

Ajouter un véhicule

Interface Vue

Controleur Front

Controleur Back

Modèle

Base de données

userSpace.php

userSpace.js

add_vehicule_controller.php

add_vehicule.php

MySQL

MongoDB

Clique sur le bouton « Ajouter un véhicule »

Informe le controleur que "Modifier" a été cliqué avec l'editField associé

Affiche une série de prompt et select pour saisir (marque, modèle, immatriculation, couleur, nombre de places, énergie (dropdown), date immatriculation (input type="date"))

Saisit les valeurs

userSpace.js

Vérifie les bon type de valeur

Envoie une requête fetch() POST vers

Appel PHP

Demande l'exécution de la requête
INSERT INTO vehicules (...) VALUES (?, ?, ?, ?, ?, ?, ?)

Confirme l'ajout

Enregistre l'action de réservation dans la collection logs

Affiche le véhicule

Affiche une alerte de succès
Recharge la page

Retourne en JSON le succès

Retour PHP

Avis



Email

Utilisateur Passager

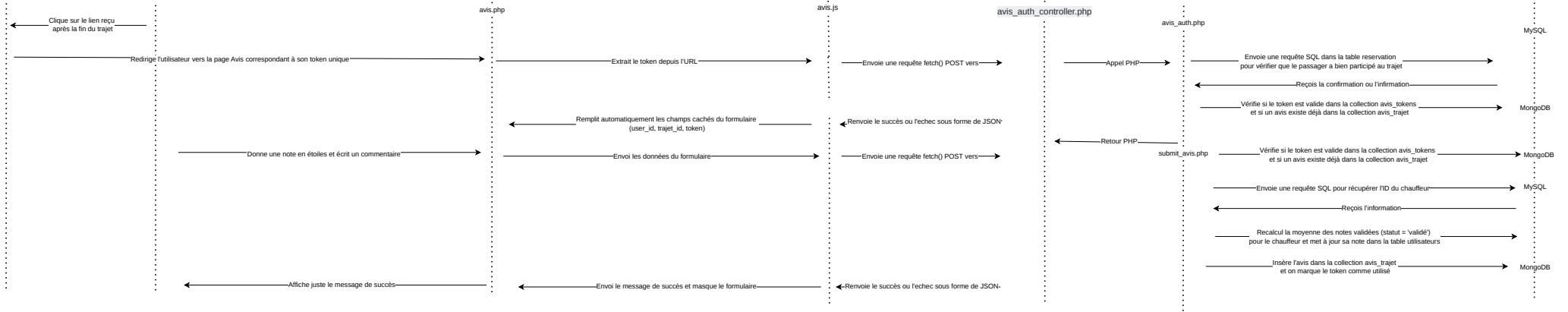
Interface Vue

Controleur Front

Controleur Back

Modèle

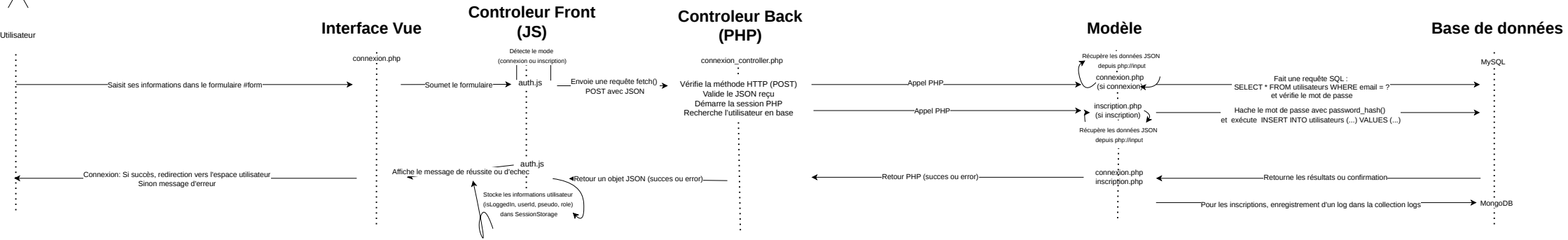
Base de données





Utilisateur

Connexion / Inscription



Gestion des trajets chauffeur



Utilisateur Chauffeur

Interface Vue

Controleur Front

Controleur Back

Modèle

Base de données

userHistory.php

get_trajets.js

get_trajets_controller.php

update_trajet.php

MySQL

MongoDB

Clique sur le bouton "Démarrer", "Arrivée" ou "Annuler"

Informe le controleur que le bouton a été cliqué

Envoie une requête fetch() POST vers

Appel PHP

Requête SQL pour mettre à jour l'état du trajet sur 'en cours', 'terminé' ou 'annulé'

Vérifie que l'utilisateur est bien le chauffeur du trajet. Si l'état du trajet doit passer sur 'terminé', on fait une requête SQL pour mettre à jour l'état de la réservation du passager sur 'réalisé'. Si l'état du trajet passe sur 'annulé', la requête passe la réservation du passager sur 'annulé'.

Confirme le bon déroulé de l'opération

On enregistre le token unique dans la collection avis_tokens

Terminé: on génère un token unique et on utilise sendMail.php pour envoyer à chaque passager un lien d'avis avec un bouton HTML personnalisé contenant le token unique.

Annulé: On utilise sendMail.php pour informer les passagers de l'annulation.

Affiche l'alerte

Envoi une alerte pour informer le succès ou l'échec de la requête

Renvoie le succès ou l'échec sous forme de JSON

Retour PHP



Utilisateur Passager

Gestion des trajets passager

Interface Vue

userHistory.php

Controleur Front

get_trajets.js

Controleur Back

get_trajets_controller.php

Modèle

annuler_reservation.php

Base de données

MySQL

Clique sur le bouton "Annuler ma réservation"

Informe le controleur que le bouton a été cliqué

Envoie une requête fetch() POST vers

Appel PHP

annuler_reservation.php
Requête SQL pour mettre à jour l'état de la table reservation sur 'annulé' et rajoute une place dans la table covoiturages.
Vérifie si une réservation existe bien
Si l'état du trajet doit passer sur 'terminé', on fait une requête SQL pour mettre à jour l'état de la réservation du passager sur 'réalisé'.
Si l'état du trajet passe sur 'annulé', la requête passe la réservation du passager sur 'annulé'

Affiche l'alerte

Envoi une alerte pour informer le succès ou l'echec de la requête

Renvoie le succès ou l'echec sous forme de JSON

Retour PHP

Confirme le succès ou l'echec de l'opération



Modifier mon espace

Utilisateur

Interface Vue

userSpace.php

Controleur Front

userSpace.js

Controleur Back

update_utilisateur_controller.php

Modèle

update_utilisateur.php

Base de données

MySQL

MongoDB

Clique sur un bouton « Modifier » à côté d'un champ (pseudo, email, mot de passe, ou rôle) →

→ Informe le modèle que "Modifier" a été cliqué avec l'editField associé →

← Affiche un prompt ou déclenche un dropdown pour entrer la nouvelle valeur ←

→ Saisit la nouvelle valeur →

→ Envoie une requête fetch() POST vers →

→ Appel PHP →

→ Demande l'exécution de la requête
UPDATE utilisateurs SET ... WHERE id = :id
Si c'est le mot de passe, application de password_hash()

← Confirme la modification ←

→ Enregistre l'action de réservation dans la collection logs →

← Affiche une alerte de succès ←

← Met à jour dynamiquement le champ modifié ←

← Retourne en JSON le succès ←

← Retour PHP ←



Participer à un trajet

Utilisateur

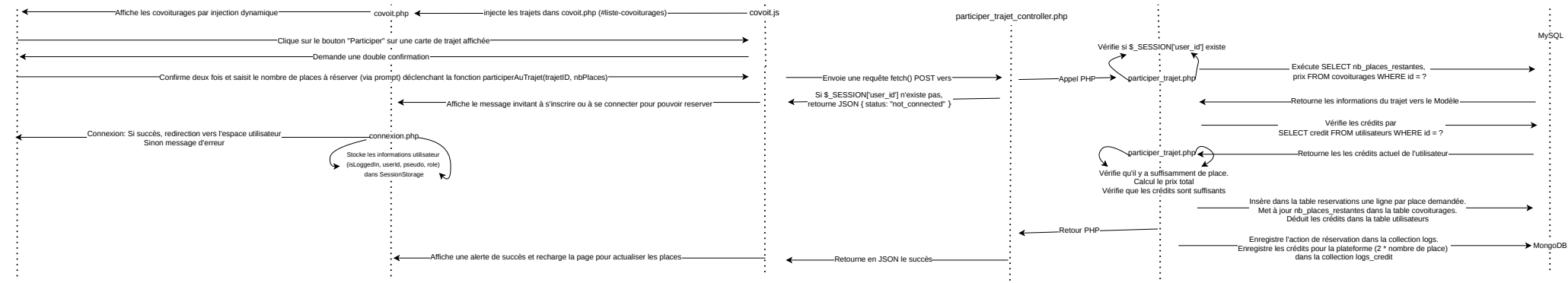
Interface Vue

Controleur Front

Controleur Back

Modèle

Base de données





Utilisateur

Programmer un trajet

Interface Vue

userHistory.php

Controleur Front

new_trajet.js

Controleur Back

get_user_controller.php

Modèle

get_user.php

Base de données

MySQL

MongoDB

Accède à l'onglet "Mes futurs trajets" dans l'espace personnel

Affiche le formulaire pour proposer un nouveau trajet

Clique sur la sélection de véhicule du catalogue et remplit les champs du formulaire

Informe le controleur que la page est chargé

Propose les véhicules sous forme d'option dans le dropdown

Une fois le bouton "Proposer" cliqué, les valeurs sont envoyés au controleur

Affiche une alerte de succès
Recharge la page

Envoie une requête fetch() POST vers

Renvoie les véhicules sous forme de JSON

new_trajet.js
Envoie une requête fetch() POST vers
Vérifie que tous les champs
sont renseignés et
que les prix est supérieur à 2

Retourne en JSON le succès

Appel PHP

add_trajet.php
Récupère les données en JSON.
Vérifie l'authentification avec \$_SESSION['user_id'].
Calcule l'heure d'arrivée grâce à l'intervalle.

Retour PHP

Requête SQL pour récupérer l'ensemble des véhicules
en lien avec l'ID de l'utilisateur

Récupère les véhicules associés à l'utilisateur

Requête SQL pour insérer le trajet dans la table covoiturages

Renvoi une confirmation

Enregistre l'action de réservation dans la collection logs



Rechercher un trajet

Utilisateur

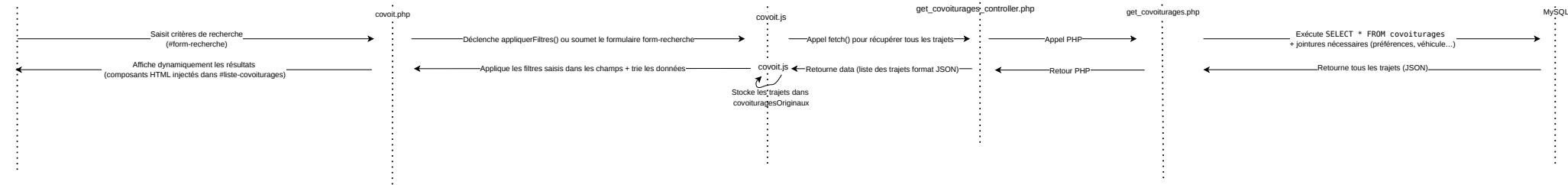
Interface Vue

Controleur Front

Controleur Back

Modèle

Base de données





Administrateur

Suspendre / Rétablir compte

Interface Vue

Controleur Front

Controleur Back

Modèle

Base de données

espace_admin.php

admin.js

suspendre_utilisateur_controller.php

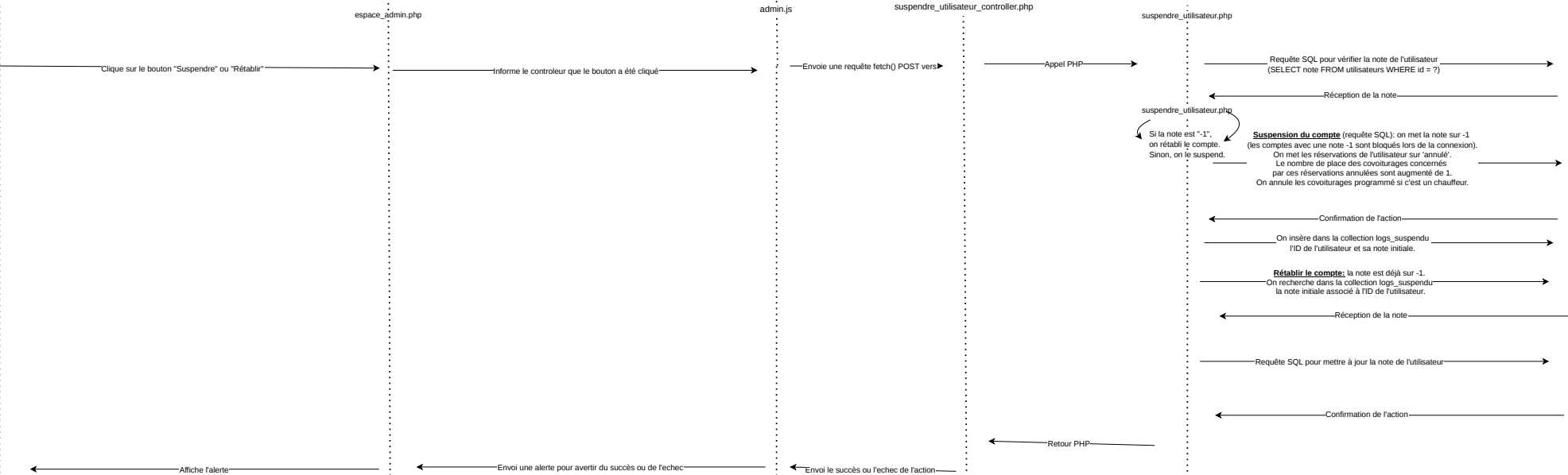
suspendre_utilisateur.php

MySQL

MongoDB

MongoDB

MySQL



Déploiement de l'application (Heroku) :

Préparation GitHub :

- Fusion de la branche de développement avec la branche principale (main)Création du dépôt, branche main, commits progressifs.
- .gitignore configuré pour exclure /vendor/, .env et Modele/data.

Configuration de Heroku :

- Création du compte.
- Création du nom de l'application.
- Dans « Deploy », connexion de Heroku avec mon compte GitHub grâce à mon lien git.
- Dans « Settings », après avoir cliqué sur « Reveal Config Vars », j'ai saisi les variables d'environnements (SMTP* pour les emails, MONGO_URI pour MongoDB Atlas, JAWSDB_URL pour la connexion avec la base de données JawsDB MySQL, DB* pour les paramètres de la base de données JawsDB MySQL et BASE_URL qui est utilisé pour le bouton intégré aux emails que les passagers reçoivent pour laisser un avis auquel je rattache un token unique).

Ajout des addons et outils externes :

- JawsDB MySQL : Dans Heroku, onglet « Ressources » j'ai ajouté l'addon « JawsDB MySQL ». Dans cette plateforme l'URL à copier dans la variable d'environnement était présent. Je l'ai donc rajouté dans la partie « Settings » / « Reveal Config Vars ».
- Installation de MySQLworkbench pour gérer la base de donnée JawsDB MySQL.
- MongoDB Atlas : J'ai créé une base de données eco_ride. J'ai copié URI de la connexion sécurisée que j'ai collé dans les variables d'environnement JawsDB MySQL. Enfin, j'ai autorisé les adresses IP à se connecter à MongoDB Atlas.

Base de données :

- Dans MySQLworkbench, connecté à JawsDB MySQL, j'ai importé ma base de données locale nommé « create_database.sql » (présent sur le dépôt GitHub).
- MongoDB : même si les collections se créent automatiquement par le code, j'ai fait la connexion entre MongoDB Atlas avec MongoDB Compass et j'y ai transféré mes collections pour avoir un historique.

Déploiement :

- Dans « Deploy », j'ai cliqué sur « Deploy Branch » configuré sur la branche git « main ». Lors des corrections, j'ai également déployé la branche « dev » et « bugfix/* » pour faire des tests.

-J'ai adapté mon code pour qu'il fonctionne aussi bien en local que déployé avec Heroku.

-Une fois la branche main déployé, j'ai effectué un test complet du site via <https://ecoride-project-9138c168add3.herokuapp.com/>

Les collections MongoDB utilisées :

- « avis_tokens » : Cette collection contient le token unique du passager et permet de vérifier si un avis à déjà été laissé.
- « avis_trajet » : Contient tous les avis, validés et refusés.
- « logs » : Contient les actions des utilisateurs (véhicules rajoutés, nouveaux membres inscrits, modification sur les comptes etc..)
- « logs_credit » : Contient les crédits retenus par la plateforme à chaque reservation.
- « logs_employe » : Contient les actions des employés.
- « logs_suspendu » : Contient les informations sur les comptes suspendus, notamment pour pouvoir les rétablir.

Déploiement avec Docker (en local) :

En parallèle du déploiement distant, j'ai également mis en place une version conteneurisée du projet via Docker et Docker Compose pour faciliter le déploiement en local avec toutes les dépendances.

J'ai défini :

- Un Dockerfile pour construire une image Apache avec PHP et les extensions nécessaires (pdo_mysql, mysqli, etc.)
- Un fichier docker-compose.yml pour orchestrer l'application web, la base MySQL et le serveur MongoDB
- Un fichier .dockerignore pour exclure les fichiers inutiles lors du build Docker
- Un fichier .htaccess pour permettre au routeur JavaScript de gérer les routes coté client en SPA (Single Page Application).

L'application peut être lancée depuis la commande :

docker-compose up --build

Et le site est accessible à l'adresse : <http://localhost:8080>

Cela permet de mettre en place un environnement de test identique pour tous les postes et simplifie la mise en production sur une plateforme compatible comme Render.com.